Phase 4

Role Request Workflow — Instructor Onboarding

PhysioGuidance — NELC-Compliant eLearning Platform

Status: Needs to be built from scratch

Assign To	1 Backend Dev + 1 Frontend Dev
Duration	2–3 days
Depends On	Phase 2 (Email module) + Phase 3 (Admin UI patterns)
Key Existing Asset	BecomeTrainerForm.tsx — 985 lines, bilingual, file uploads (submit = console.log)

1. How Each Role Gets Created

Understanding the role assignment model is essential before building the workflow:

Role	How It's Assigned	By Whom
SUPER_ADMIN	Seed script only — never created via UI	System
ADMIN	SUPER_ADMIN assigns directly in User Management	SUPER_ADMIN
INSTRUCTOR	User applies via 'Become Trainer' form → Admin reviews → Approves	User + Admin
COURSE_MANAGER	Admin assigns directly from User Management	ADMIN
FINANCE_MANAGER	Admin assigns directly from User Management	ADMIN
SUPPORT	Admin assigns directly from User Management	ADMIN
STUDENT	Automatic on registration (auth.service.ts line 72)	System

Key Insight: INSTRUCTOR is the ONLY role that requires an application and approval process. All other roles are either system-assigned (STUDENT, SUPER_ADMIN) or directly assigned by admins.

2. Application Flow Diagram

The complete lifecycle of an instructor application:

Step	Action	API / Component	Notes
①	Student fills 'Become Trainer' form	BecomeTrainerForm.tsx	Student Dashboard
②	Files uploaded to GCP Storage	POST /api/storage/upload	Returns file URLs
③	Application submitted	POST /api/role-requests	Status: PENDING
④	Admin notified via email	new-application-admin template	Badge count updates
⑤	Admin reviews application	/admin/role-requests/[id]	Full profile + downloads
⑥	Admin decides	approve / reject / request-info	See outcomes below

Decision Outcomes:

■ **APPROVE**
- role_requests.status = 'APPROVED'
- authService.assignRole(userId, 'INSTRUCTOR')
- Send approval email
- User can switch to Instructor dashboard

■ REJECT
- role_requests.status = 'REJECTED'
- Set reviewNote with reason
- Send rejection email
- User can reapply (new request)

■ MORE INFO
- role_requests.status = 'MORE_INFO'
- Set reviewNote with question
- Send more-info email
- User edits original request (PATCH)

3. Database Schema

New table: role_requests

`backends/src/database/schema/role-requests.ts`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Auto-generated
userId	UUID	FK → users ON DELETE CASCADE	Applicant
requestedRole	VARCHAR(50)	NOT NULL	e.g. 'INSTRUCTOR'
status	VARCHAR(20)	DEFAULT 'PENDING'	PENDING / APPROVED / REJECTED / MORE_INFO
profileData	JSONB	NOT NULL	fullNameAr, fullNameEn, nationality, profession, specialty, qualification, rank, scfhsNumber, gender, phone, twitter, linkedin
bankData	JSONB	—	accountHolderName, bankName, iban
resumeFiles	JSONB	—	[{ type: 'education'|'experience'|'cv'|'certificates', url, filename }]
reviewedBy	UUID	FK → users (nullable)	Admin who reviewed
reviewNote	TEXT	nullable	Rejection reason / info request / notes
reviewedAt	TIMESTAMP	nullable	When admin reviewed
createdAt	TIMESTAMP	DEFAULT NOW()	Submission date
updatedAt	TIMESTAMP	DEFAULT NOW()	Last modification

profileData JSONB Structure (matches BecomeTrainerForm fields):


```
{
  "fullNameArabic": "■■■■",
  "fullNameEnglish": "Mohammed",
  "email": "m@example.com",
  "countryCode": "+966",
  "mobileNumber": "5XXXXXXXX",
  "nationality": "Saudi",
  "profession": "Physiotherapist",
  "specialty": "Sports Rehabilitation",
  "qualification": "Master",
  "academicRank": "Assistant Professor",
  "hasScfhsNumber": true,
  "scfhsNumber": "XXXXX",
```

```
  "gender": "male",
  "twitter": "@handle",
  "linkedIn": "linkedin.com/in/..."
}
```

Migration: After creating the schema file, run: `make db-generate && make db-migrate`

4. API Endpoints

Method	Endpoint	Auth Guard	Purpose
POST	/api/role-requests	@Roles('STUDENT')	Submit new application
GET	/api/role-requests/mine	Authenticated	View my applications + status
GET	/api/role-requests	ADMIN / SUPER_ADMIN	List all (filter, paginate)
GET	/api/role-requests/:id	ADMIN / SUPER_ADMIN	Full application detail
PATCH	/api/role-requests/:id/approve	ADMIN / SUPER_ADMIN	Approve → auto-assign role
PATCH	/api/role-requests/:id/reject	ADMIN / SUPER_ADMIN	Reject with reason
PATCH	/api/role-requests/:id/request-info	ADMIN / SUPER_ADMIN	Ask for more info
PATCH	/api/role-requests/:id	@Roles('STUDENT')	Resubmit after MORE_INFO

Validation Rules:

- **One PENDING request per role per user** — prevent duplicate submissions
- **Cannot apply for a role you already have** — return 409 Conflict
- **Can reapply after rejection** — creates a new role_request record
- **MORE_INFO resubmit** — user edits the ORIGINAL request via PATCH (not a new submission)
- **POST guard: @Roles('STUDENT')** — only students can apply
- **Not logged in?** — redirect to signup/login, then back to the form

POST /api/role-requests — Request Body:

```
{
  "requestedRole": "INSTRUCTOR",
  "profileData": { ... },     // See JSONB structure above
  "bankData": { ... },
  "resumeFiles": [
    { "type": "cv", "url": "https://storage.../cv.pdf", "filename": "CV.pdf" },
    { "type": "education", "url": "...", "filename": "Degree.pdf" }
  ]
}
```

GET /api/role-requests — Query Parameters:

Param	Type	Default	Description
status	string	all	Filter: PENDING, APPROVED, REJECTED, MORE_INFO, or all
page	number	1	Page number
limit	number	20	Items per page
search	string	""	Search by applicant name/email

5. Backend Module Structure

New files to create:

File	Purpose
backends/src/modules/role-requests/role-requests.module.ts	NestJS module: imports, controllers, providers
backends/src/modules/role-requests/role-requests.service.ts	Business logic: create, list, approve, reject, request-info
backends/src/modules/role-requests/role-requests.controller.ts	HTTP endpoints with guards and validation
backends/src/modules/role-requests/dto/create-role-request.dto.ts	Validation: profileData, bankData, resumeFiles
backends/src/modules/role-requests/dto/review-role-request.dto.ts	Validation: reviewNote (required for reject/info)

Modified files:

File	Change
backends/src/app.module.ts	Register RoleRequestsModule in imports
frontend/src/components/BecomeTrainerForm.tsx	Replace console.log with real API call
frontend/src/lib/api.ts	Add roleRequestsApi methods

Frontend API additions (frontend/src/lib/api.ts):

```
export const roleRequestsApi = {
  submit: (data) => post('/api/role-requests', data),
  getMine: () => get('/api/role-requests/mine'),
  getAll: (params) => get('/api/role-requests', { params }),
  getById: (id) => get(`/api/role-requests/${id}`),
  approve: (id, note?) => patch(`/api/role-requests/${id}/approve`, { note }),
  reject: (id, note) => patch(`/api/role-requests/${id}/reject`, { reviewNote: note }),
  requestInfo: (id, note) => patch(`/api/role-requests/${id}/request-info`, { reviewNote: note }),
  resubmit: (id, data) => patch(`/api/role-requests/${id}`, data),
};
```

---

6. Email Notification Templates

4 new templates using the Email module built in Phase 2:

Template File	Trigger	Recipient	Content
application-approved.template.ts	Admin approves	Applicant	'Your trainer application has been approved! You can now switch to your Instructor dashboard.'

Template File	Trigger	Recipient	Content
application-rejected.template.ts	Admin rejects	Applicant	'Your application was not approved. Reason: {reviewNote}. You may reapply.'
application-more-info.template.ts	Admin requests info	Applicant	'We need more information about your application: {reviewNote}'
new-application-admin.template.ts	Student submits	All ADMINs	'New instructor application from {name}. Review it in the admin dashboard.'

7. Frontend Pages & UI

7A. Wire Become Trainer Form

`frontend/src/components/BecomeTrainerForm.tsx`

- **Replace console.log** submit handler with real API flow
- **Upload flow:** Upload files to POST /api/storage/upload first → get URLs → include in role-request body
- Show **loading state** during upload + submission
- Success: **'Application submitted! We'll review it within X days.'**
- If user has PENDING application → show **status card** instead of form
- If user is already INSTRUCTOR → show **'You're already a trainer'** message
- If user is NOT logged in → **redirect to /signup or /signin** then back to form

---

7B. Student Application Status Page

`frontend/src/app/[locale]/dashboard/student/applications/page.tsx`

Status	Visual	Message	Action
PENDING	Yellow badge	'Your application is under review'	Show submission date
APPROVED	Green badge	'Approved! Switch to Instructor dashboard'	Show 'Switch Role' button
REJECTED	Red badge	'Not approved' + admin's reason	Show 'Reapply' button
MORE_INFO	Orange badge	'Additional information needed' + admin's note	Show resubmit form / edit button

- Timeline of status changes (submitted → reviewed → approved/rejected)
- Add **'My Applications'** link to student sidebar navigation

---

7C. Admin Role Requests List Page

`frontend/src/app/[locale]/dashboard/admin/role-requests/page.tsx`

- Data table columns: **Applicant Name, Email, Requested Role, Status, Submitted Date, Actions**
- Filter by status: Pending / Approved / Rejected / All
- **Badge count** showing pending applications in admin sidebar nav
- Click row → navigate to full review page
- Server-side pagination with search

7D. Admin Application Review Page

`frontend/src/app/[locale]/dashboard/admin/role-requests/[id]/page.tsx`

- **Applicant profile:** photo, name (AR/EN), credentials, specialty, qualification, rank, SCFHS number
- **Resume files:** download links for each PDF (CV, education, experience, certificates)
- **Bank info:** account holder, bank name, IBAN

- **Social links:** Twitter, LinkedIn
- **Action buttons:**

Button	UX Flow	Backend Effect
■ Approve (green)	Confirmation modal → calls PATCH .../approve	Auto-assigns INSTRUCTOR role + sends email
■ Reject (red)	Modal with reason textarea (required) → calls .../reject	Sends rejection email with reason
■ Request Info (orange)	Modal with message textarea → calls .../request-info	Sends more-info email

- Status history timeline at the bottom of the page

7E. Sidebar Navigation Updates

Dashboard	Change	Details
Student Sidebar	Add 'My Applications' link	Show status indicator dot (yellow/green/red)
Admin Sidebar	Add 'Role Requests' link	Show pending count badge (e.g., '3')

8. Complete File Summary

New Files (13)

File	Owner	Type
backends/src/database/schema/role-requests.ts	Backend	Schema
backends/src/modules/role-requests/role-requests.module.ts	Backend	Module
backends/src/modules/role-requests/role-requests.service.ts	Backend	Service
backends/src/modules/role-requests/role-requests.controller.ts	Backend	Controller
backends/src/modules/role-requests/dto/create-role-request.dto.ts	Backend	DTO
backends/src/modules/role-requests/dto/review-role-request.dto.ts	Backend	DTO
backends/src/.../templates/application-approved.template.ts	Backend	Email
backends/src/.../templates/application-rejected.template.ts	Backend	Email
backends/src/.../templates/application-more-info.template.ts	Backend	Email
backends/src/.../templates/new-application-admin.template.ts	Backend	Email
frontend/.../dashboard/student/applications/page.tsx	Frontend	Page
frontend/.../dashboard/admin/role-requests/page.tsx	Frontend	Page
frontend/.../dashboard/admin/role-requests/[id]/page.tsx	Frontend	Page

Modified Files (3)

File	Change
backends/src/app.module.ts	Register RoleRequestsModule
frontend/src/components/BecomeTrainerForm.tsx	Wire submit to real API
frontend/src/lib/api.ts	Add roleRequestsApi methods

9. Verification Checklist

#	Test Case	Expected Result	Pass
1	Student fills Become Trainer form	Application saved as PENDING in DB	■
2	Student views application status	Status card shown in student dashboard	■

#	Test Case	Expected Result	Pass
3	Duplicate PENDING submission	Blocked with error message	■
4	Admin sees new application	Listed in Role Requests + badge count	■
5	Admin opens application	Full profile, resume downloads, bank info	■
6	Admin approves	INSTRUCTOR role auto-assigned + email sent	■
7	Approved user switches role	Instructor dashboard loads correctly	■
8	Admin rejects with reason	User sees reason + 'Reapply' button	■
9	Admin requests more info	User sees note + edit/resubmit form	■
10	Already-INSTRUCTOR visits form	'You're already a trainer' message shown	■
11	Non-logged-in user visits form	Redirected to signup/login, then back	■
12	Admin gets email on new application	Email received with applicant details	■

10. Team Assignment & Parallel Work

Developer	Tasks	Day 1	Day 2	Day 3
Backend Dev	4.1, 4.2, 4.3, 4.8	Schema + migrations + service + controller	Email templates + storage integration + testing	Bug fixes + edge cases
Frontend Dev	4.4, 4.5, 4.6, 4.7, 7E	Wire BecomeTrainerForm + student status page	Admin list + review pages	Sidebar updates + AR/EN + testing

Coordination point: Backend dev finishes the API endpoints on Day 1. Frontend dev can start wiring the form on Day 1 using mock data, then switch to real API on Day 2 once endpoints are ready.